FUNCTION REFERENCE

Understanding cstats.R

A practical guide to the data setup, aggregation, statistical tests, and plots provided by the cstats R script.

Based on the functions in `cstats.R`. The helper `cstats.parse` is intentionally excluded.

At a glance

The functions form a workflow: read a table, tell cstats which columns define the experiment and outcomes, optionally average technical replicates, then run analyses or make plots. The setup functions store their selections as attributes on the data frame: `F` for factors, `V` for measured variables, and `R` for replicate identifiers.

1. `cstats.read` — import a delimited file
2. `cstats.set_factors` — designate experimental factors
3. `cstats.set_vars` — designate measured outcomes
4. `cstats.set_reps` — designate replicate identifiers
5. `cstats.aggregate` — average measurements within groups
6. `cstats.2wanova` — two-factor ANOVA for every selected outcome
7. `cstats.tukey` — Tukey-adjusted comparisons between levels of factor 2
8. `cstats.wilcox` — pairwise Wilcoxon rank-sum tests
9. `cstats.t` — pairwise Welch two-sample t-tests
10. `cstats.barplot.tukey` — means, SD, observations, and Tukey annotations
11. `cstats.barplot.wilcox` — means, SD, observations, and Wilcoxon annotations
12. `cstats.barplot.t` — means, SD, observations, and t-test annotations

> **Important:** Several analyses require the factor and variable attributes to be set first. For two-factor tests and all three plotting functions, factor order matters: the first factor defines groups/panels; the second factor is the one compared within each group.

cstats.read

```
cstats.read(path, separator = "\t")
```

Purpose. Reads a delimited text file as a data frame. The default separator is a tab, which is suitable for tab-separated files; pass another delimiter such as `","` for comma-separated data.

Arguments. `path` is the file path. `separator` is the one-character field delimiter.

What it does. Uses `read.delim` with a header row, then applies the script's comma-decimal conversion helper. Any existing cstats metadata attributes (`F`, `V`, `R`) are removed from the imported table.

Returns. The imported data frame, without cstats selections set.

```
data <- cstats.read("training/tutorial.txt")
data <- cstats.read("data.csv", separator = ",")
```

cstats.set_factors

```
cstats.set_factors(df, factors)
```

Purpose. Records the names of columns that define the experimental groups or conditions. Later functions read these names from the data frame's `F` attribute.

Arguments. `df` is the data frame; `factors` is a character vector of column names (for example, `c("Time", "Treatment")`).

Validation and return. Stops with an error if any requested name is not a column of `df`. Otherwise returns the same data frame with its `F` attribute set.

```
data <- cstats.set_factors(data, c("Time", "Treatment"))
```

> For two-factor analyses, the order you supply is meaningful: the first factor is used as the grouping/stratifying factor and the second as the within-group comparison factor.

cstats.set_vars

```
cstats.set_vars(df, vars)
```

Purpose. Declares which columns contain quantitative outcomes to analyze or aggregate. The selected names are stored in the `V` attribute.

Arguments. `df` is the data frame; `vars` is a character vector of outcome-column names.

Validation and return. Stops if a requested name is not present in the data frame; otherwise returns the data frame with `V` set.

```
data <- cstats.set_vars(data, c("Viability", "Marker"))
```

Aggregation takes means of these columns, and `cstats.2wanova` fits a separate model for each. The single-outcome test and plot functions also require their `var` argument to be one of the selected variables.

cstats.set_reps

```
cstats.set_reps(df, replicates)
```

Purpose. Declares column(s) identifying replicate units, storing their names in the `R` attribute. These columns are used as grouping keys during aggregation; they are not replicate counts.

Arguments. `df` is the data frame; `replicates` is a character vector naming replicate identifier columns.

Validation and return. Stops if a name is missing from the data frame; otherwise returns the data frame with `R` set.

```
data <- cstats.set_reps(data, "BioRep")
```

Choose identifiers that preserve the independent experimental units. The function checks column names, but it does not determine whether the chosen replicate definition is scientifically appropriate.

cstats.aggregate

```
cstats.aggregate(df)
```

Purpose. Averages each selected outcome over rows sharing the same factor and replicate identifiers. This is intended to combine technical replicates while keeping biological replicate units distinct.

Requirements. Factor (`F`) and outcome (`V`) attributes must be set. If replicate identifiers (`R`) are not set, it prints a warning and groups only by factors, so all rows within each factor combination are averaged together.

Returns. A plain data frame with one row per group and mean values for selected outcomes. The factor, outcome, and replicate attributes are carried onto the result.

```
data_agg <- cstats.aggregate(data)
```

> **Design check:** Without replicate identifiers, biological replicates are also averaged together. Set `R` correctly before aggregation if those independent units must remain separate for downstream analysis.

cstats.2wanova

```
cstats.2wanova(df)
```

Purpose. Fits a two-way linear model with an interaction for every outcome in `V`, using the two factors in `F`.

For each outcome, the model has the form `outcome ~ factor1 * factor2`. In R, `*` includes both main effects and their interaction. The function calls `lm` and then base R's `anova` on each fit.

Requirements. Both attributes must be present, and exactly two factors must be selected.

Returns. A named list of ANOVA tables, one table per outcome. Inspect each table for the factor and interaction terms.

```
anova_results <- cstats.2wanova(data_agg)
anova_results$Viability
```

> The function uses base R's sequential ANOVA table; interpretation depends on model assumptions, design, and factor ordering. It does not check those assumptions or apply a multiple-testing adjustment across outcomes.

cstats.tukey

```
cstats.tukey(df, var)
```

Purpose. Compares levels of the second factor within each level of the first factor, using estimated marginal means from a two-way ANOVA model with interaction.

Arguments. `df` must have exactly two selected factors and at least one selected outcome. `var` names the outcome column to analyze.

Adjustment and return. Pairwise contrasts are adjusted using Tukey's method. Returns the `emmeans` pairwise-comparison result (not a list over all outcomes); it includes contrasts and inferential statistics such as estimates and adjusted p-values.

```
tukey_results <- cstats.tukey(data_agg, "Viability")
summary(tukey_results)
```

The implementation calls the selected outcome directly; unlike some other test/plot functions, it does not explicitly check that `var` is listed in the `V` attribute.

cstats.wilcox

```
cstats.wilcox(df, var, base = NULL, altrn = "two.sided")
```

Purpose. Runs unpaired Wilcoxon rank-sum tests (the Mann–Whitney U test) for every unordered pair of levels.

Selected factors	Comparisons performed
One factor	All pairs of levels of that factor.
Two factors	All pairs of levels of factor 2, restricted to the specified level of factor 1.

Arguments. `var` must be a selected outcome. With two factors, `base` must be a level of factor 1. `altrn` is passed to `wilcox.test` and can be `"two.sided"`, `"less"`, or `"greater"`.

Returns. A list of `wilcox.test` results, named for each comparison. Comparisons with fewer than two observations in either group are skipped with a message. The function does not adjust p-values across comparisons.

```
wilcox_results <- cstats.wilcox(
  data_agg, "Viability", base = "48"
)
```

> Samples are treated as independent, not paired. Use an analysis that matches the study design; the test does not correct for multiple comparisons.

cstats.t

```
cstats.t(df, var, base = NULL, altrn = "two.sided")
```

Purpose. Runs pairwise unpaired Welch two-sample t-tests for a selected outcome. The comparison scheme is the same as `cstats.wilcox`: all pairs for one factor, or pairs of factor-2 levels within a chosen factor-1 level for two factors.

Arguments. `var` must be in `V`. With two factors, `base` must be a level of factor 1. `altrn` selects a two-sided or one-sided alternative.

Why Welch? The function calls `t.test` without setting `var.equal = TRUE`, so R uses Welch's method, which does not assume equal population variances.

Returns. A named list of test results. Comparisons with fewer than two observations per group are skipped with a message. There is no multiple-comparison adjustment.

```
t_results <- cstats.t(
  data_agg, "Viability", base = "48"
)
```

> The tests are unpaired and assume independent observations. Ensure that the rows supplied represent the correct experimental units.

cstats.barplot.tukey

```
cstats.barplot.tukey(df, var)
```

Purpose. Creates one plot for each level of factor 1. Each plot shows factor-2 group means, mean ± standard deviation error bars, and jittered individual observations.

Statistics and annotations. Fits the same two-factor interaction model used by `cstats.tukey` and calculates Tukey-adjusted pairwise comparisons of factor 2 within each factor-1 level. Significant comparisons are marked with brackets and stars.

Requirements and return. Exactly two factors must be set, and `var` must be in `V`. Returns a named list of ggplot objects, with one entry per factor-1 level. Display a plot with `plots[[1]]` or by its list name.

```
plots <- cstats.barplot.tukey(data_agg, "Viability")
plots[[1]]
```

Star cutoffs are p < 0.05 (*), p < 0.01 (**), and p < 0.001 (***); nonsignificant contrasts are not annotated. The adjusted Tukey p-values are used.

cstats.barplot.wilcox

```
cstats.barplot.wilcox(df, var)
```

Purpose. Produces one plot per level of factor 1. Within each plot it shows means, standard deviations, and jittered raw observations for the levels of factor 2.

Statistics and annotations. Runs an unpaired, two-sided Wilcoxon rank-sum test for each pair of factor-2 levels within each factor-1 level. It annotates significant comparisons with brackets and stars.

Requirements and return. Exactly two factors must be set, and `var` must be in `V`. Returns a named list of ggplot objects, one for each factor-1 level.

```
plots <- cstats.barplot.wilcox(data_agg, "Viability")
plots[[1]]
```

> These plots summarize data with means and SD even though the significance tests are rank-based. The pairwise p-values are not adjusted for multiple comparisons; interpret stars accordingly.

cstats.barplot.t

```
cstats.barplot.t(df, var)
```

Purpose. Produces one plot per factor-1 level, showing factor-2 means, mean ± SD error bars, and jittered individual observations.

Statistics and annotations. Runs an unpaired, two-sided Welch t-test for every pair of factor-2 levels within each factor-1 level. Significant comparisons receive brackets and stars.

Requirements and return. Exactly two factors must be set, and `var` must be in `V`. Returns a named list of ggplot objects, one per factor-1 level.

```
plots <- cstats.barplot.t(data_agg, "Viability")
plots[[1]]
```

> The plot uses an unadjusted t-test p-value for each pair, with no multiple-comparison correction. Tests are unpaired and assume independent observations.

Putting the workflow together

This example reads the training table, identifies two experimental factors, an outcome, and biological replicate identifiers, then averages technical replicates before analysis.

```
data <- cstats.read("training/tutorial.txt")
data <- cstats.set_factors(data, c("Time", "Treatment"))
data <- cstats.set_vars(data, "Viability")
data <- cstats.set_reps(data, "BioRep")

data_agg <- cstats.aggregate(data)

anova_results <- cstats.2wanova(data_agg)
tukey_results <- cstats.tukey(data_agg, "Viability")
plots <- cstats.barplot.tukey(data_agg, "Viability")
plots[[1]]
```

Interpretation reminders

- Choose factors, measured outcomes, and replicate identifiers to match the experimental design.
- Aggregation averages outcomes within factor-and-replicate groups; it does not validate the experimental unit.
- The t-test and Wilcoxon functions compare independent groups and do not adjust for multiple comparisons.
- Plot error bars show standard deviations, not standard errors or confidence intervals.
- Check assumptions, sample sizes, and the suitability of each statistical test before drawing conclusions.

Function behavior described here reflects the current implementation in `cstats.R`. This guide explains the functions and is not a substitute for statistical advice.